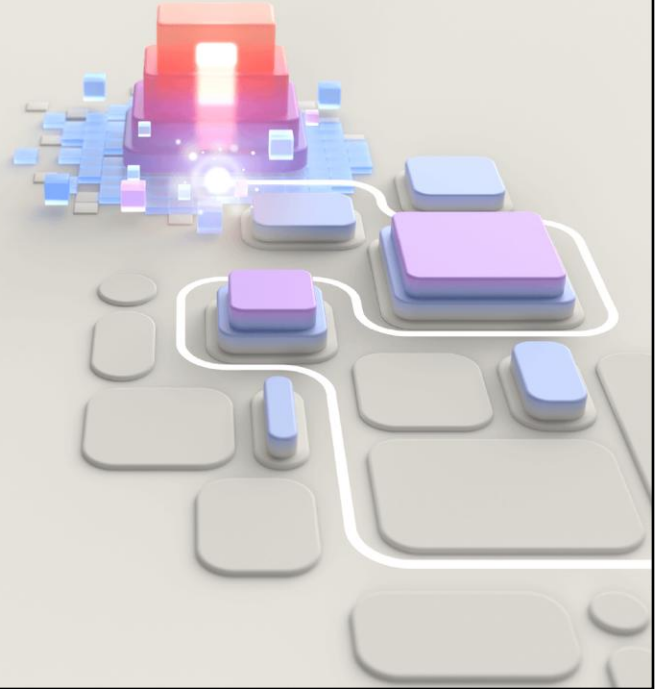




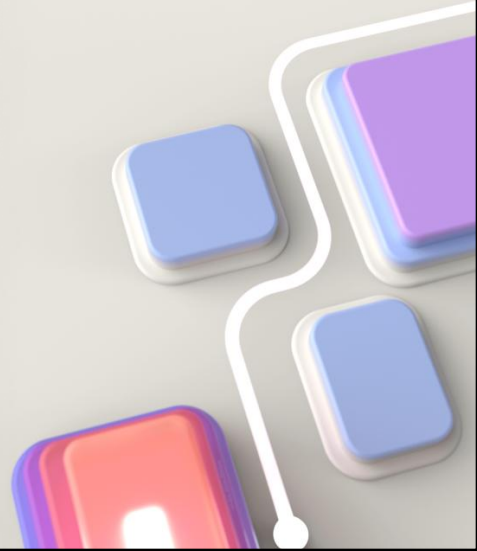
PL-400T00 Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.



Introduction to developing with Power Platform

© Copyright Microsoft Corporation. All rights reserved.



This is an introductory learning path for the remainder of the course
Be careful not to go into too much detail in these modules.

Modules



- Introduction to Power Platform developer resources
- Use developer tools to extend the Power Platform
- Introduction to extending the Microsoft Power Platform
- Work with Dataverse APIs

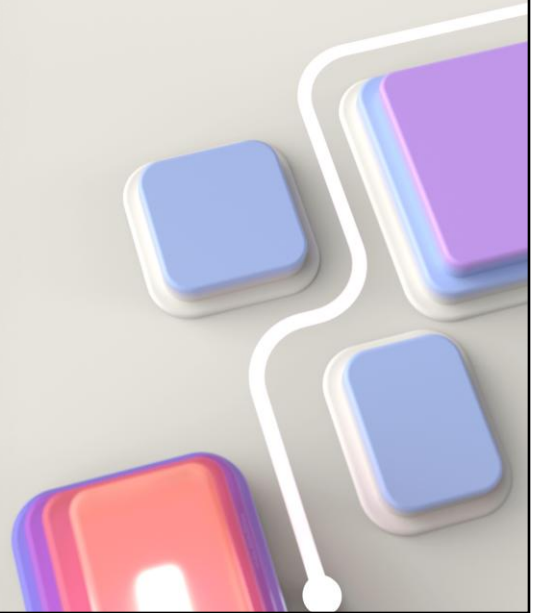
© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Introduction to Microsoft Power Platform developer resources
2. Use developer tools to extend Microsoft Power Platform
3. Introduction to extending Microsoft Power Platform
4. Work with Dataverse APIs

Module 1: Introduction to Power Platform developer resources

© Copyright Microsoft Corporation. All rights reserved.



The goal of this module is to give a broad-scoped overview of the developer experience as it relates to Microsoft Power Platform. It will cover a high-level introduction how the ecosystem is represented, with the intended audience being a software developer with limited experience in working with Microsoft Power Platform.

Learning Objectives

After completing this module, you will be able to:

- 1 Explain what solution components exist within Microsoft Power Platform.
- 2 Explain key components of Microsoft Dataverse and the Common Data Model.
- 3 Explain what Azure solution elements relate to Microsoft Power Platform.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Introduction to Microsoft Power Platform developer resources

<https://learn.microsoft.com/training/modules/introduction-developing-power-platform>



© Copyright Microsoft Corporation. All rights reserved.

Introduction to Microsoft Power Platform developer resources

<https://learn.microsoft.com/training/modules/introduction-developing-power-platform>

As a Power Platform developer, you need to understand the gaps that exist between what can be accomplished through configuration versus code. Sometimes, existing features might not provide the functionality needed to meet a requirement, and Dataverse provides various points where developers can extend the common functionality by using code.

To identify gaps where features don't already exist, it's crucial that all Power Platform developers familiarize themselves with the capabilities of Dataverse.

Typically, any code-customization activities are referred to as "extending" a model-driven application. Therefore, if you notice terms like extensibility, it generally means code in terms of model-driven Power App development. Within the extensibility model, activities are grouped into two broader buckets: extending the user experience and extending the Dataverse platform.

To extend the user experience of a model-driven Power App, you can use web resources. Model-driven Power Apps expose a JavaScript client API that allows you to interact with application pages.

Because TypeScript, which compiles down to JavaScript, is a recommended tooling to use in many scripting scenarios, both languages are generalized into the term of "client script." With client script, you can configure form event handlers in the same way that you would an HTML form element. You can also call script functions from within a command bar (ribbon) and can consume client script web resources from within HTML web resources.

HTML web resources are used to display custom presentation logic within a form when more advanced requirements exist that can't be achieved by using the standard out-of-the-box forms.

Extending the Dataverse platform is required whenever some level of automation needs to occur that existing features do not exist to support the current activities. These types of extensibility points run as server-side code through an asset called a plug-in. Power Automate and other options enable asynchronous automation and logic, but they haven't been able to do everything that plug-ins currently do in terms of synchronous rules. When requirements for synchronous operations exist within a model-driven application, plug-ins are still required.

Plug-in development is an essential skill and one of the most common tasks that is required of a Power Apps developer.

Extending with Azure covered in LP 10

Explore the Power Platform Developer Guide

A wealth of assets is available that developers can review to better acquaint themselves on developing against the Dataverse framework.

Sample code

<https://github.com/microsoft/PowerApps-Samples>

<https://github.com/pnp/powerapps-samples>

Useful links

Microsoft Dataverse developer documentation

<https://learn.microsoft.com/power-apps/developer/data-platform>

Model-driven apps developer

<https://learn.microsoft.com/power-apps/developer/model-driven-apps>

Power Apps component framework

<https://learn.microsoft.com/power-apps/developer/component-framework/overview>

© Copyright Microsoft Corporation. All rights reserved.

A wealth of assets is available that developers can review to better acquaint themselves on developing against the Dataverse framework. Some of these are legacy assets that are branded as Dynamics 365 SDK documentation. Dataverse is an evolution of the Dynamics 365 platform. Many of the existing materials surrounding how to extend the platform still apply. While these materials can be great resources, they're no longer current; therefore, use them with discretion.

Microsoft also provides a large amount of sample code that shows you a host of activities that you would commonly perform as a Dataverse developer. These activities include interacting with the Web API or Organization Service, and also how to take advantage of a useful collection of assemblies called XrmTooling for accelerating common actions such as authentication against the Dataverse by means of third-party applications. These codes can be found in GitHub and in various other mechanisms, such as through NuGet packages. You can find links to these in the following section.

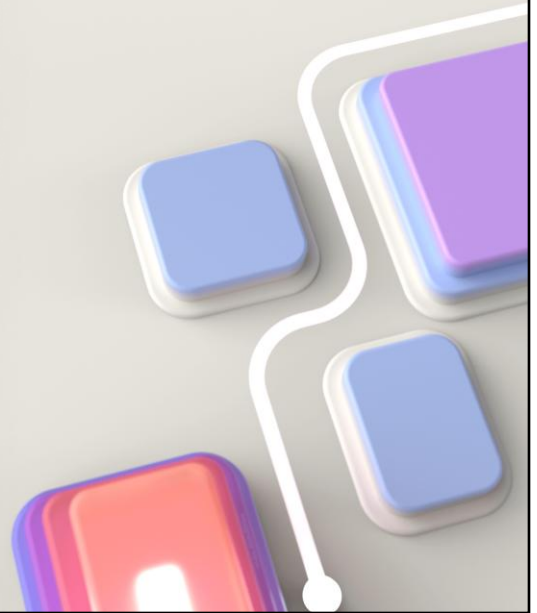
The following are links that should be added as favorites to any Power Apps developer's repository.

<https://github.com/microsoft/PowerApps-Samples>

<https://github.com/pnp/powerapps-samples>

Module 2: Use developer tools to extend the Power Platform

© Copyright Microsoft Corporation. All rights reserved.



This module will focus on the available developer tools that can help you perform development activities with Microsoft Power Platform.

Learning Objectives

After completing this module, you will be able to:

- 1 Work with the Configuration Migration tool
- 2 Work with Package Deployer
- 3 Run the Plugin Registration Tool
- 4 Work with GitHub Copilot

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Use developer tools to extend Microsoft Power Platform

<https://learn.microsoft.com/training/modules/use-developer-tools-extend>

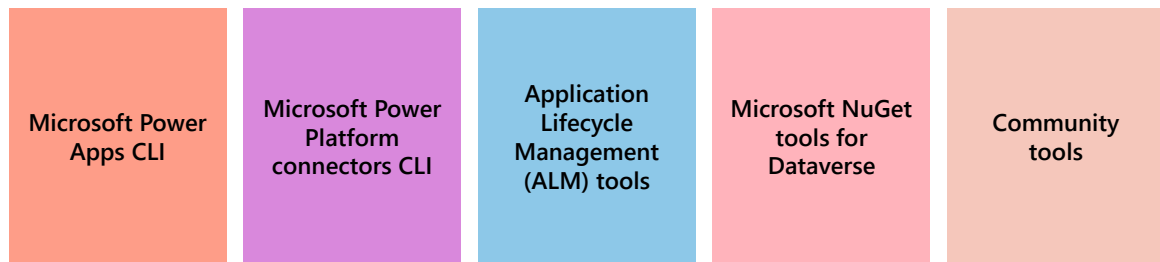


© Copyright Microsoft Corporation. All rights reserved.

Use developer tools to extend Microsoft Power Platform

<https://learn.microsoft.com/training/modules/use-developer-tools-extend>

Introduction to developer Microsoft Power Platform tooling



© Copyright Microsoft Corporation. All rights reserved.

Developers can use many of the traditional development tools when building solutions for Microsoft Power Platform. But there are various distinct tools exclusively targeting Microsoft Power Platform development. Microsoft provides official tooling via [NuGet package manager](#) and other published tool installers. There is also a vibrant community of tools and libraries complementing what Microsoft publishes. In this module, we will explore the tools with which a Microsoft Power Platform developer should be familiar.

Microsoft Power Apps CLI

The Power Apps CLI is a command-line interface to create code components. For example, this is how you would initialize and manage the lifecycle of a Power Apps Component Framework component. The tool also can help with developing plug-ins as well. The tool is solution-aware and can help you deploy the supported platform extensions to a development environment for testing.

Microsoft Power Platform connectors CLI

The connector CLI provides a command-line tool to create, download, update, and validate custom connectors. The tool is essential for advanced editing of the custom connector definition. You also must use the tool if you are preparing a custom connector for open-sourcing or certification.

Application Lifecycle Management (ALM) tools

An important part of implementing an application lifecycle management for your projects is having a repeatable process that allows you to modify the solution components and have these changes recorded in a source control system. While the steps to accomplish these tasks can be performed manually, having an automated process is better to ensure consistency and repeatability. To support building these automations Microsoft provides build tasks for Azure DevOps and Actions for GitHub.

Review the [complete list](#) of the Azure DevOps build tasks for more specifics on each task.

Review the [complete list](#) of GitHub Actions for more specifics on each action.

Microsoft NuGet tools for Dataverse

Which we will cover in the next slide

Community tools

There are many community tools, libraries, and solutions, most of them open source, available to improve Microsoft Power Platform developer experience and fill in the gaps in the official Microsoft development tools and libraries. Some of the tools are listed on [Community tools for Microsoft Dataverse](#) page.

Tools created by the community are not supported by Microsoft. If you have questions or issues with

community tools, contact the publisher of the tool.

Microsoft NuGet tools

Microsoft provides a variety of tools to help support development against Dataverse, which are available through [NuGet packages](#) and the Power Platform CLI.



© Copyright Microsoft Corporation. All rights reserved.

- Code Generation Tool: Generates early-bound .NET Framework classes that represent the table data model that is used by model-driven apps.
- Configuration Migration Tool: Tool that is used to move configuration data across Dataverse environments.
- Package Deployer: Tool that enables administrators to deploy packages in a Dataverse environment. Developers combine solution assets and custom code in a package to be deployed by Package Deployer. Independent software vendors (ISVs) must use the same package format when preparing their solutions for publishing on AppSource.
- PluginRegistrationTool: Tool that is used to register custom code plug-in assemblies and steps in a Dataverse environment.
- Solution Packager Tool: Tool that can disassemble Dataverse solutions into multiple XML files to be managed by a source control system. It can also be used to rebuild a Dataverse solution from the source XML files.

Use GitHub Copilot



- Pair programming tool, designed to empower and improve developer productivity
- Compatible with all programming languages used by the Power Platform (C#, JavaScript, TypeScript etc.)
- Requires a paid GitHub plan:
 - Copilot Individual
 - Copilot Business
 - Copilot Enterprise
- Always review / validate output independently

```
1 function createContact(applicationContext) {  
2   //Get the form context from the application context  
3  
4   //Get the Account ID from the form context  
5  
6   //Create a contact object with static values, including the accountId property  
7  
8   //Actually create the contact - show an error to the user if it fails  
9 }
```

© Copyright Microsoft Corporation. All rights reserved.

GitHub Copilot is an AI coding assistant that helps you write code faster and with less effort, allowing you to focus more energy on problem solving and collaboration. Copilot offers coding suggestions as you type: sometimes the completion of the current line, sometimes a whole new block of code. You can accept all, or part, of a suggestion, or you can ignore the suggestion and keep typing.

Using the chat feature, you can ask Copilot how best to solve a problem. Or you can ask Copilot to explain someone else's code. If your code has a bug, you can ask Copilot how to fix it.

As with any AI tool, mistakes can sometimes occur. It's therefore vital that developers review the output of any code generated by AI to confirm that it's valid, does not reference deprecated components or coding patterns that are discouraged.

Learning Path 3 practice labs



Introduction to developing with the Power Platform

Lab 4: Power Platform tools

© Copyright Microsoft Corporation. All rights reserved.

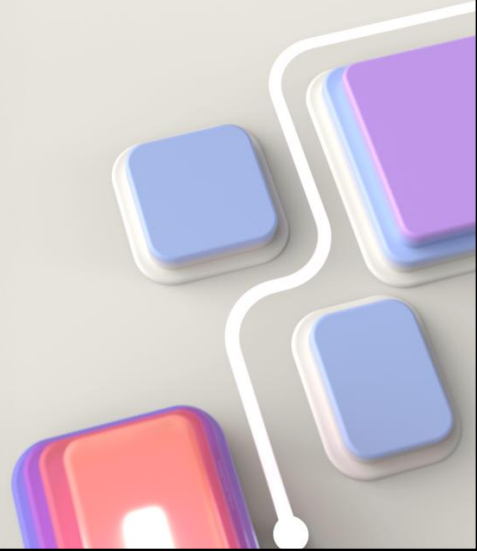
As we continue to build our solution, we will now install and configure tools used for extending the Power Platform with code.

- Configuration Migration Tool
- XrmToolBox

Other tools such as Visual Studio should be preinstalled

Module 3: Introduction to extending the Microsoft Power Platform

© Copyright Microsoft Corporation. All rights reserved.



This module will focus on the underlying solution architecture from a technical perspective and what extensibility options exist. It will also cover the ever-important element of Microsoft Power Platform development, which is the decision-making process of determining when to use configuration versus code.

Learning Objectives

After completing this module, you will be able to:

- 1** Identify which elements architecturally comprise Microsoft Power Platform.
- 2** Learn about the areas of extensibility that are available to customize Microsoft Power Platform through code.
- 3** Discover different approaches to common business scenarios in respect to achieving extensibility with configuration versus code.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Introduction to extending Microsoft Power Platform

<https://learn.microsoft.com/training/modules/introduction-power-platform-extensibility-model>



© Copyright Microsoft Corporation. All rights reserved.

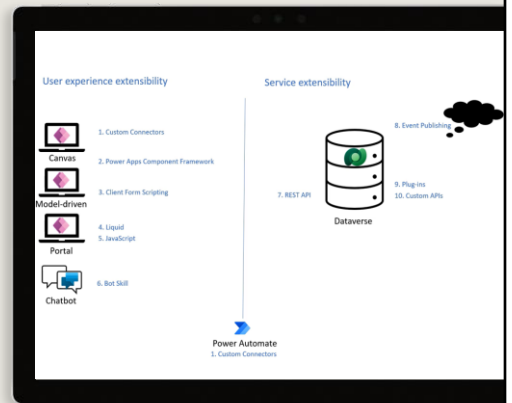
Introduction to extending Microsoft Power Platform

<https://learn.microsoft.com/training/modules/introduction-power-platform-extensibility-model>

Demo

Extensibility points

The extensibility points of Microsoft Power Platform components are purposely implemented at various points to allow custom code to be used when low code approaches aren't possible.



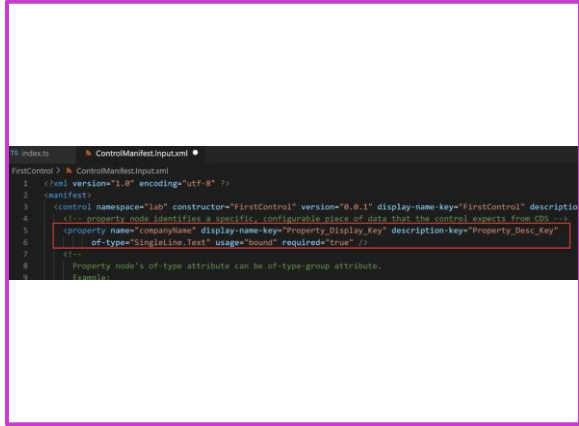
© Copyright Microsoft Corporation. All rights reserved.

Play this video <https://www.microsoft.com/en-us/videooplayer/embed/RWBJuM?postJsIIMsg=true>

<https://www.microsoft.com/en-us/videooplayer/embed/RWBJuM>

Power Apps code component framework (PCF)

Code components are implemented using HTML, CSS, and TypeScript.



© Copyright Microsoft Corporation. All rights reserved.

For many years, HTML web resources used to be a go-to extensibility mechanism for the client-side in model-driven apps. One of the weaknesses of this approach was low reuse of these monolithic and non-extensible elements. Now HTML web resources have been replaced by PCF controls.

PCF allows developers to create reusable components that can be used by makers instead of the standard controls. This is a good example of when advances in the platform capabilities allow business to focus their development efforts on creating solid infrastructure and enabling app makers.

Code components are implemented using HTML, CSS, and TypeScript. While not required that you use any particular UI Framework, [React](#) is a popular choice.

Client scripting

Client scripting allows you to use JavaScript in model-driven apps to implement business logic

- Form load
- Data in a column changes
- Form is saved

© Copyright Microsoft Corporation. All rights reserved.

Client scripting allows you to use JavaScript in Power Apps model-driven apps to implement business rules. Client scripting should be used as an alternative when declarative business rules do not meet the requirements. Client scripting runs on a model-driven form in response to the following form events:

- Form Load
- Data in a column changes
- Form is saved

Using the Dataverse APIs

Dataverse provides two styles of APIs that developers can use to interact with data

- **Web API** – The Web API is available at an OData v4 RESTful endpoint. Use this for any programming language that supports HTTP requests and authentication using OAuth 2.0.
- **Organization Service** – The Organization Service is a .NET Framework SDK with .NET assemblies provided by Microsoft along with typed class generators for table classes. When you access data from a plugin this is how you access data. The service is instantiated and available to the plugin code without the need to authenticate.

© Copyright Microsoft Corporation. All rights reserved.

Dataverse provides two styles of APIs that developers can use to interact with data: Web API and Organization Service. The following provides a high-level overview of each:

Web API - The [Web API](#) is available at an OData v4 RESTful endpoint. Use this for any programming language that supports HTTP requests and authentication using OAuth 2.0. You can find examples [here](#).

Organization Service - The [Organization Service](#) is a .NET Framework SDK with .NET assemblies provided by Microsoft along with typed class generators for table classes. When you access data from a plugin this is how you access data. The service is instantiated and available to the plugin code without the need to authenticate. The service is also available for usage on the Windows client. You can find examples [here](#).

Both APIs support their own approaches to building data queries in addition to supporting [FetchXML](#). FetchXML is a proprietary query language that is used in Dataverse. The FetchXML language allows complex queries to be crafted across related tables and use Dataverse-specific conditions and operators. The Power Automate Dataverse connector also supports FetchXML.

Building plugins



Plugins are .NET classes that implement a **IPlugin** interface provided by the Dataverse SDK assemblies



Implement business logic within the event pipeline

© Copyright Microsoft Corporation. All rights reserved.

Plugins are .NET classes that implement a **IPlugin** interface provided by the Dataverse SDK assemblies. That interface requires you implement only one method named **Execute**. The following is an example of a minimal implementation:

```
public sealed class MyFirstPlugin : IPlugin { public void Execute(IServiceProvider serviceProvider) { //Business Logic } }
```

The **Execute** method has one parameter of type **IServiceProvider**. This interface has one method **GetService** that you can use to retrieve services available to a plugin. The following are examples of the services available:

IPluginExecutionContext - this gives you access to the message being processed and information about the requestor.

ITracingService - this gives you access to write to the trace log for diagnostic purposes.

IOrganizationServiceFactory - this gives you access to retrieve an **OrganizationService** for use in accessing data from the plug-in.

The following is an example of using **GetService** to get the execution context:

```
IPluginExecutionContext context = (IPluginExecutionContext) serviceProvider.GetService(typeof(IPluginExecutionContext));
```

Using **context.InputParameters** can give you access to the originating message and **context.OutputParameters** can give you access to what is being returned to the caller.

A good understanding of the execution context is fundamental for a plugin developer. You can learn more about the execution context [here](#).

To have a plugin execute, it must be registered to run for a specific message. This can be accomplished using the Plugin Registration tool.

Custom APIs



Operations in Dataverse are defined as messages



Custom APIs create new messages in the event pipeline



Write a plug-in to perform the logic for the custom API

© Copyright Microsoft Corporation. All rights reserved.

Operations in Dataverse are defined as messages. Custom APIs offer a code-first way to define new messages that you can extend Dataverse web services. These messages can then be invoked, just like the system messages, but to execute custom business logic. For example, if you have specific requirements to search for a customer using a predefined sequence of API calls then, instead of having each API caller implement that sequence, you could implement a *findcustomer* custom API. The *findcustomer* API would now implement the required logic to search for the customer and return the results. This would ensure that instead of each app having to figure out how to find a customer and doing it inconsistently they could call the custom API that would process the search requests the same way on each occasion. And if any changes are required, they would need to be implemented in one place only.

To define a new custom API, you start by creating a custom API record. This can be done through the maker portal, through code, or using Dataverse solutions. As part of creating the record you identify the custom API name, and the request and response parameters.

To implement the logic of the custom API, you create a plugin and register it on the Main Operation stage of the pipeline. Custom API implementation is the only scenario where a plugin can be registered on the Main Operation stage. You will see an example of doing this later in the exercise. You also cannot implement a plug-in and allow the custom API to be used to trigger other automation. For example, creating a Custom API that contains data about an event that occurred in another system would allow that system to call that API in Dataverse, where many different integration points with Azure, or Web Hooks, or Power Automate, or asynchronous plug-ins on the API will be able to initiate other automations.

Determine when to configure or when to code

As a developer, you should approach apps on Microsoft Power Platform from the perspective that writing code for achieving desired business application functionality should be considered as an exception to no-code and low-code approaches

© Copyright Microsoft Corporation. All rights reserved.

As a developer, you should approach apps on Microsoft Power Platform from the perspective that writing code for achieving desired business application functionality should be considered as an exception to no-code and low-code approaches. However, quality areas such as maintainability, upgradability, stability, and performance should also be considered when you are determining what is the best approach for a given scenario.

Learning what Microsoft Power Platform can do out of the box is important because you do not want to build something that they already do and just needs to be configured. That also applies to the functionality implemented in the available Dynamics 365 apps. For example, invoice calculations using variable multi-currency price lists are complex, but they are well implemented, and time-tested in Dynamics 365 Sales. If this functionality is required, you should consider using Dynamics 365 Sales built-in capabilities instead of implementing your own calculation engine.

You should also recognize that Microsoft Power Platform often implements something in a particular way that benefits the platform. This may be different if you are used to doing it in custom application code. It is not uncommon for developers that are new to building Microsoft Power Platform solutions to try to customize the platform the way they used to build custom apps previously. This should be avoided when possible and you should try to take advantage of what the platform does well, rather than try to change it to what you are used to doing. For example, in the past you may have implemented column-level security using custom code. This is, however, not required in Dataverse where column-level security is an out-of-the-box feature and simply needs to be configured.

Business rules vs. client script

The advantage of business rules is that they are easy to understand and implement for a non-developer, and they can be included as part of a Dataverse solution for deployment in production.

© Copyright Microsoft Corporation. All rights reserved.

The advantage of business rules is that they are easy to understand and implement for a non-developer, and they can be included as part of a Dataverse solution for deployment in production. In the organizations where developer skills are not easily available and application lifecycle management is not implemented, business rules would be a preferred way to implement logic even if some workarounds are required, for example, intermediate calculated fields holding the values to check in a rule.

However, at some point, business rules are not capable of implementing the business requirements, or maybe the complexity that the rules cause developers to prefer writing the logic in client script. One scenario might be if you have a complex nested "if/then/else" logic that would be better achieved in a *switch* statement or simple sequence of *if* blocks. The other one is when you are dealing with dynamic values that are not readily accessible by a business rule. For example, form notifications and filtering out choice values are not available through business rules.

Workflows/Power Automate flows vs. plugins

Dataverse offers various methods to implement logic to respond to events in the system, in particular to the data changes such as create, update, delete of the data rows.

Capability	Power Automate cloud flow	Dataverse classic workflow	Plugin
Synchronous or Asynchronous	Asynchronous	Either (Power Automate flows are recommended instead of asynchronous workflows)	Either
Access External Data	Yes, using connectors	No	Yes, using APIs, some security restrictions
Maintenance	Makers	Business Users	Developers
Can Run As	Current user or flow owner	Current user or workflow owner	Any licensed user, system, or current user
Can Run On Demand	Yes	Yes	No
Can Nest Child Processes	Yes	Yes	Yes
Execution Stage	After	Before/After	Before/After
Triggers	Create, Column Change, Delete, On Demand, Scheduled	Create, Column Change, Delete, On Demand	Create, Column Change, Delete, other messages including custom

© Copyright Microsoft Corporation. All rights reserved.

Dataverse offers various methods to implement logic to respond to events in the system, in particular to the data changes such as create, update, delete of the data rows. Business requirements can be satisfied by no-code solutions using workflow and Power Automate and by extending Dataverse using either server-side (plugins) or client-side (script) code.

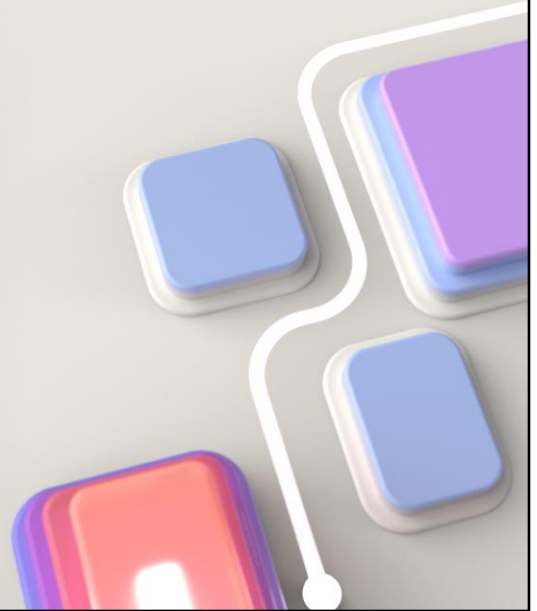
You can evaluate the options using an approach similar to the one used in the business rules vs. client script discussion: evaluate business requirements and organization capabilities to implement them. There are also some fundamental differences in capabilities. The following table can help you determine when it might be better to use one or another approach.

Microsoft Power Platform continuously evolves and as a developer you should be aware of new and upcoming features. For example, if your solution required custom settings or configuration values, previously you would have had to use custom tables to store these values and then use custom code or platform tools to deploy the data. The new addition of environment variables simplified this task and reduced it to declaring your variables, including them as part of the solutions, and setting the values, all without any code.

Low code plug-ins offer a viable alternative for .NET plugins and should be considered

Module 4: Work with Dataverse APIs

© Copyright Microsoft Corporation. All rights reserved.



Learning Objectives

After completing this module, you will be able to:

- 1 Interact with the Dataverse Web API.
- 2 Authorize against Dataverse with OAuth.
- 3 Use OData to query data.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Work with Dataverse Web API

<https://learn.microsoft.com/training/modules/common-data-service-web-api>



© Copyright Microsoft Corporation. All rights reserved.

Work with Dataverse Web API

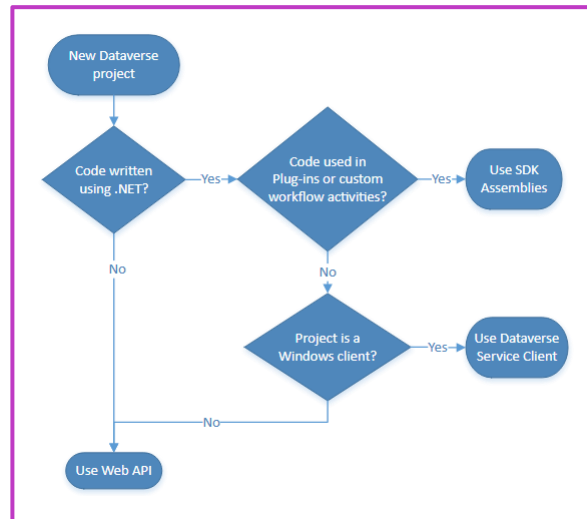
Web API vs. the Organization Service

Web API

- REST
- OData 4.0 compliant

Organization Service

- .NET Required for Plug-ins
- Dataverse service client for Windows client applications



© Copyright Microsoft Corporation. All rights reserved.

Two main ways for interacting with data and metadata in Microsoft Dataverse are the Web API and the Organization Service. You can use the Organization Service when writing plug-ins or workflow extensions by using the .NET Framework SDK assemblies that are provided by Microsoft.

The following graphic is a decision tree that can help you determine when to use the Web API versus the Organization Service (through SDK Assemblies) and also when to use XRM Tooling Assemblies (available from Microsoft) for Windows applications.

OData 4.0

Dataverse Web API is implemented as an OData (OASIS Open Data Protocol) service. By using an industry standard such as this, you're allotted all the benefits of open standard development, such as cross-platform compatibility and simple implementation against a common, predefined protocol.

Dataverse doesn't provide specific assembly tooling for the Web API; instead, we recommend that you use community libraries that conform to the OData protocol. If you want more information about which libraries exist, go to <https://www.odata.org/libraries/>.

OData provides the ability to interact with Dataverse data by means of CRUD operations that are available through standard GET, POST, PATCH, and DELETE HTTP methods (more information about these methods will be provided later in this module). You can also perform almost any other operation that is exposed through Dataverse [Event Framework](#) by invoking an OData *function* or *action* that has the same name as its corresponding Organization Service message. CRUD-related messages are the only ones that are not available because those messages are achieved by means of the previously-described standard HTTP methods.

REST

Fundamentally, OData protocol is a set of RESTful (**RE**presentational **S**tate **T**ransfer) APIs, a time-tested industry standard for providing interoperability between systems. REST consists of six guiding principles that restrict the way that a service can process and respond to requests to ensure that non-functional properties of that service are met (such as performance, scalability, simplicity, reliability, and so on).

For the purposes of this lesson, you should keep in mind that OData conforms to these constraints and that the Common Data Web API in turn also conforms to the constraints because it implements the OData 4.0 protocol.

FetchXML

FetchXML is a robust query language that allows for complex querying of Dataverse data. Microsoft provides a way for users to run FetchXML queries within a Web API query. This approach can be valuable if you encounter a scenario where FetchXML is a better fit for your particular query versus using OData query syntax. More information about FetchXML is provided in a later unit in this module.

Organization service



NET Framework (CRM SDK) or .NET Core (Dataverse SDK)



Add SDK assemblies using NuGet



IOrganizationService Interface



Early vs Late binding

© Copyright Microsoft Corporation. All rights reserved.

The Organization web service provides access to the business data, data table definitions (metadata), and data operations supported by the Dataverse platform. The Organization service is one of two Dataverse web services, the other being the Discovery service used to access Dataverse environment details.

The Organization service is accessed from .NET Framework or .NET Core based applications by using provided classes in the Dataverse SDK for .NET assemblies. For .NET Framework only development, the SDK assemblies are available in the Microsoft.CrmSdk.CoreAssemblies NuGet package. For .NET Framework or .NET Core development, the SDK assemblies are available in the Microsoft.PowerPlatform.Dataverse.Client NuGet package.

The Organization service is accessed from .NET Framework or .NET Core based applications by using provided classes in the Dataverse SDK for .NET assemblies. For .NET Framework only development, the SDK assemblies are available in the Microsoft.CrmSdk.CoreAssemblies NuGet package. For .NET Framework or .NET Core development, the SDK assemblies are available in the Microsoft.PowerPlatform.Dataverse.Client NuGet package.

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/overview>

<https://learn.microsoft.com/power-apps/developer/data-platform/sdk-client-transition>

<https://learn.microsoft.com/en-us/power-apps/developer/data-platform/work-with-data>

IOrganizationService Methods

The IOrganizationService interface has 8 methods:

- Create
- Retrieve
- Update
- Delete
- RetrieveMultiple
- Associate
- Disassociate
- **Execute**

© Copyright Microsoft Corporation. All rights reserved.

The Organization service exposes only the Execute method. The other methods in the IOrganizationService interface are simply wrappers around the Execute method. These other methods are provided for convenience. You can perform all operations using only the Execute method.

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/iorganizationservice-interface>

Early bound vs Late bound

Using the Organization service

Late bound

- **Entity** class
- Refer to tables and columns as strings
- Run time validation

Early bound

- Generate set of classes for each table
- Intellisense in Visual Studio
- Compile time validation

© Copyright Microsoft Corporation. All rights reserved.

When you work with the Organization service assemblies you have two programming styles you can use: late-bound and early-bound.

The key difference between early and late binding involves type conversion. While early binding provides compile-time checking of all types so that no implicit casts occur, late binding checks types only when the object is created or an action is performed on the type. The Entity class requires types to be explicitly specified to prevent implicit casts.

Late binding allows you to work with custom tables (entities) or columns (attributes) that weren't available when your code was compiled.

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/early-bound-programming>

Create a row

```
var account = new Entity("account");
account["name"] = "Contoso";
account["credithold"] = false;
account["numberofemployees"] = 500;
account["revenue"] = new Money(new decimal(5000000.00));
account["accountcategorycode"] = new OptionSetValue(1); //Preferred customer
Guid accountid = svc.Create(account);
```

© Copyright Microsoft Corporation. All rights reserved.

Example using late bound to create an account record

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/entity-operations-create>

Retrieve a row

```
Entity accountEntity = svc.Retrieve(  
    "account",  
    accountid,  
    new ColumnSet("name", "accountnumber"));
```

© Copyright Microsoft Corporation. All rights reserved.

Example using late bound to retrieve an account record

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/entity-operations-retrieve>

Querying with the Organization service

Different methods for querying data

- QueryExpression
- QueryByAttribute
- FetchExpression (with FetchXML)
- LINQ

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/entity-operations-query-data>

Batching requests

ExecuteMultipleRequest

- ContinueOnError = true/false
- ReturnResponses = true/false

Maximum batch size = 1000 requests

© Copyright Microsoft Corporation. All rights reserved.

```
ExecuteMultipleRequest requestWithResults = new
ExecuteMultipleRequest()
{
    Settings = new ExecuteMultipleSettings()
    {
        ContinueOnError = false,
        ReturnResponses = true
    },
    Requests = new OrganizationRequestCollection()
};
EntityCollection input =
GetCollectionOfEntitiesToCreate();
foreach (var entity in input.Entities)
{
    CreateRequest createRequest = new CreateRequest {
    Target = entity };
    requestWithResults.Requests.Add(createRequest);
}
ExecuteMultipleResponse responseWithResults =
(ExecuteMultipleResponse)service.Execute(requestWithResults);
```

Can batch requests to reduce network traffic

Decision on whether to continue if a request fails or to continue with the batch

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/execute-multiple-requests>

<https://learn.microsoft.com/power-apps/developer/data-platform/webapi/execute-batch-operations-using-web-api>

Transactions

ExecuteTransactionRequest

- ReturnResponses = true/false

Maximum batch size = 1000 requests

```
ExecuteTransactionRequest requestWithResults = new
ExecuteTransactionRequest ()
{
    Settings = new ExecuteMultipleSettings()
    {
        ReturnResponses = true
    },
    Requests = new OrganizationRequestCollection()
};
EntityCollection input =
GetCollectionOfEntitiesToCreate();
foreach (var entity in input.Entities)
{
    CreateRequest service. Execute = new CreateRequest {
    Target = entity };
    requestWithResults.Requests.Add(createRequest);
}
ExecuteTransactionResponse responseWithResults =
(ExecuteTransactionResponse)service.Execute(requestWithR
esults);
```

© Copyright Microsoft Corporation. All rights reserved.

Can perform a batch as a single transaction
If there is an error all requests are rolled back

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/use-execute-transaction>

Concurrency

Optimistic Concurrency

Use UpdateRequest

Set ConcurrencyBehavior

- IfRowVersionMatches
- AlwaysOverwrite
- Default

```
Id = new Guid("a976763a-ba1c-e811-a954-000d3af451d6");
Entity retrievedAccount = svc.Retrieve("account",
Id, Columnset(true));
var account = new Account();
account.Id = retrievedAccount.Id;
account.RowVersion = retrievedAccount.RowVersion;
account.CreditOnHold = true;
var request = new UpdateRequest()
{
    Target = account,
    ConcurrencyBehavior =
ConcurrencyBehavior.IfRowVersionMatches
};
try
{
    svc.Execute(request);
}
```

© Copyright Microsoft Corporation. All rights reserved.

Cannot lock records in Dataverse

Can use Optimistic concurrency to check if records has been updated since it was read and then to abort or overwrite

Note: Cannot use Update and must use UpdateRequests

<https://learn.microsoft.com/power-apps/developer/data-platform/optimistic-concurrency>

<https://learn.microsoft.com/power-apps/developer/data-platform/org-service/entity-operations-update-delete?tabs=late#optimistic-concurrency-behavior>

API Limits

Service Protection limits

- Evaluated over a 5-minute rolling window
- Combination of number of requests, execution time, and number of concurrent requests
- Handle 429 error (Too many requests)

API limits

- Evaluated over 24 hours
- Based on license
- Can purchase more API calls

© Copyright Microsoft Corporation. All rights reserved.

Service protection limits cannot be changed, and you need to handle Error 429 and retry after the duration specified in Retry-After

<https://learn.microsoft.com/power-apps/developer/data-platform/api-limits>

Dataverse Web API



OData v4 RESTful endpoint



Uses OAuth 2.0 for authentication



Microsoft provides MSAL library for OAuth 2.0 authentication



Verbs such as POST, GET, and PATCH

© Copyright Microsoft Corporation. All rights reserved.

The Dataverse Web API provides a development experience that can be used across a wide variety of programming languages, platforms, and devices. The Web API implements the OData (Open Data Protocol), version 4.0, an OASIS standard for building and consuming RESTful APIs over rich data sources.

The Dataverse Web API provides a development experience that can be used across a wide variety of programming languages, platforms, and devices. The Web API implements the OData (Open Data Protocol), version 4.0, an OASIS standard for building and consuming RESTful APIs over rich data sources.

Microsoft's Dataverse uses OAuth 2.0 as its authorization standard. OAuth provides an industry-standardized mechanism for authenticating client applications against a resource.

<https://learn.microsoft.com/power-apps/developer/data-platform/webapi/overview>

<https://learn.microsoft.com/azure/active-directory/develop/msal-overview>

Authentication vs. authorization

Authentication is the process or action of verifying the identity of a user or process.

Microsoft's solution to this verification process is Entra ID. Entra ID supports a number of options to verify the identity of a user or process.

Abstracting your identity provider allows for a good separation of concerns because managing usernames and passwords can be a difficult (and risky) process.

Authorization is the process or action of verifying whether an authenticated user is authorized to access the resources that are being provided.

Presently, authorization to Dataverse is at the tenant level, and permissions are delegated to the application based on the current signed-in user.

Therefore, you won't use OAuth to govern app-level security, which is instead handled by means of the Power Apps admin center.

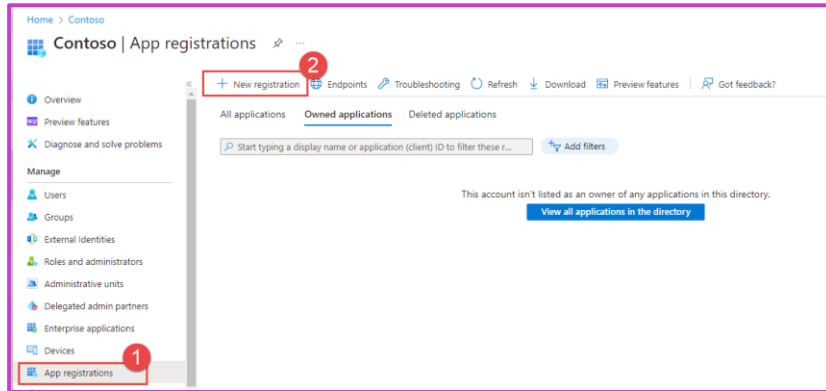
© Copyright Microsoft Corporation. All rights reserved.

Authentication is the process or action of verifying the identity of a user or process. Microsoft's solution to this verification process is Azure Active Directory (Azure AD). Azure AD supports a number of options to verify the identity of a user or process. Abstracting your identity provider allows for a good separation of concerns because managing usernames and passwords can be a difficult (and risky) process.

Authorization is the process or action of verifying whether an *authenticated* user is *authorized* to access the resources that are being provided. Presently, authorization to Dataverse is at the tenant level, and permissions are delegated to the application based on the current signed-in user. Therefore, you won't use OAuth to govern app-level security, which is instead handled by means of the Power Apps admin center.

If you want to learn more about the concepts of authentication and authorization, go to the Azure Active Directory section of Microsoft Docs: [Authentication basics](#).

Register Dataverse apps with Azure AD



To successfully connect to your Dataverse, you must first register an app with Entra ID, which can be completed in the Azure portal.

© Copyright Microsoft Corporation. All rights reserved.

To successfully connect to your Dataverse, you must first register an app with Entra ID, which can be completed in the [Azure portal](#). Depending on the type of app that you want to make, a few different settings are available for you to configure (web apps versus native apps that are installed natively on a device). For more information on settings that are required for each of these types, see [Types of app registration](#).

To register an app with Entra ID, you can go to the **App Registrations** section of the Azure Active Directory menu blade and then select **New Registration**.

Register Dataverse apps with Entra ID

Specify the name of your app and what type of account access you need.

If you're registering a web app, specify a Redirect URI by going to the Authentication section, setting the type to Web, and then entering a redirect URI.

Register an application

* Name
The user-facing display name for this application (this can be changed later).

MS Learn Sample App

Supported account types
Who can use this application or access this API?

☐ Accounts in this organizational directory only (Paritta Group only - Single tenant)

☒ Accounts in any organizational directory (Any Azure AD directory - Multitenant)

☐ Accounts in any organizational directory (Any Azure AD directory - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

[Help me choose...](#)

Redirect URI (optional)
We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Web

© Copyright Microsoft Corporation. All rights reserved.

Specify the name of your app and what type of account access you need. If you're registering a web app, specify a Redirect URI by going to the **Authentication** section, setting the type to **Web**, and then entering a redirect URI.

The following list summarizes when to use the different account types:

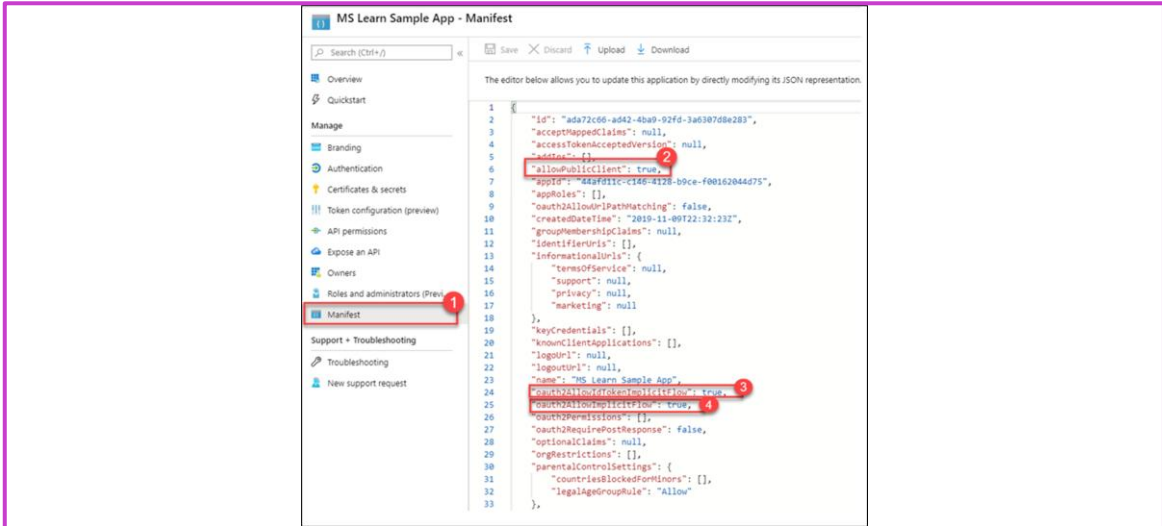
- **Accounts in this organizational directory only (Single tenant)**
 - All user and guest accounts in your directory can use your application or API.
 - *Use this option if your target audience is internal to your organization.*
- **Accounts in any organizational directory (Any Entra ID directory - Multitenant)**
 - All users with a work or school account from Microsoft can use your application or API, including schools and businesses that use Microsoft 365.
 - *Use this option if your target audience is business or educational customers and to enable multitenancy.*
- **Accounts in any organizational directory (Any Azure AD directory - Multitenant) and personal Microsoft accounts (for example, Skype and Xbox)**

- All users with a work, school, or personal Microsoft account can use your application or API. It includes schools and businesses that use Microsoft 365 and personal accounts that are used to sign in to services like Xbox and Skype.

For this example, `https://callbackurl` was entered for the **Redirect URI** because you will be accessing with Postman in a later lesson, which is a public application. Note this URL because you will need it to connect to Dataverse successfully.

Depending on the complexity of your application setup, you might want to configure other authentication settings. Refer to the Azure Active Directory documentation for steps on how to complete this task.

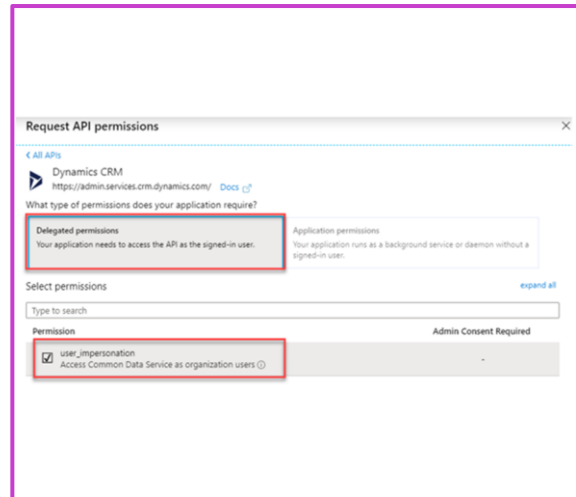
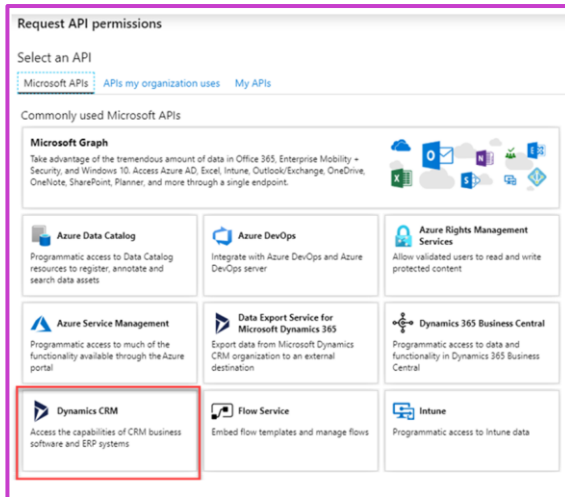
Entra ID App manifest



© Copyright Microsoft Corporation. All rights reserved.

You will also need to set a few flags to **true** in the app's manifest to allow access from Postman and other external applications. This access is only being granted for demonstration purposes only. We recommend that you read the [Azure Active Directory app manifest](#) documentation to fully understand the implications of these flags.

API permissions



© Copyright Microsoft Corporation. All rights reserved.

Next, go to the **API permissions** tab and ensure that your application is granted user impersonation access to your Dataverse environment.

You should now have an app endpoint that can successfully connect to your Dataverse environment.

Use OData to query data

Create rows

OData

```
POST [Organization URI]/api/data/v9.2/accounts HTTP/1.1
Content-Type: application/json; charset=utf-8
OData-MaxVersion: 4.0
OData-Version: 4.0
Accept: application/json

{
  "name": "Sample Account",
  "creditonhold": false,
  "address1_latitude": 47.639583,
  "description": "This is the description of the sample account",
  "revenue": 5000000,
  "accountcategorycode": 1
}
```

© Copyright Microsoft Corporation. All rights reserved.

OData is a time-tested language for querying RESTful APIs. This lesson explores a few ways of performing CRUD operations against Dataverse data.

To create rows, use the HTTP POST method.

See <https://learn.microsoft.com/en-us/power-apps/developer/data-platform/webapi/insomnia?view=dynamics-ce-odata-9> for using Insomnia to demo the web API

Use OData to query data

Retrieve rows

OData

```
GET [Organization URI]/api/data/v9.2/accounts(00000000-0000-0000-0000-000000000001)
```

© Copyright Microsoft Corporation. All rights reserved.

To retrieve rows, use the HTTP GET method.

The following sample retrieves an account with the ID of 00000000-0000-0000-0000-000000000001:

```
GET [Organization URI]/api/data/v9.2/accounts(00000000-0000-0000-0000-000000000001)
```

Other methods are available for querying data by using standard OData query syntax. For more information on this process, see [Retrieve an entity record using the Web API](#).

See <https://learn.microsoft.com/en-us/power-apps/developer/data-platform/webapi/insomnia?view=dynamics-ce-odata-9> for using Insomnia to demo the web API

Use FetchXML to query data

FetchXML is a robust query language that was developed by Microsoft to enable complex operations to be performed against Dataverse data.

FetchXML can be used by both

- Dataverse Web API
- Organization Service

© Copyright Microsoft Corporation. All rights reserved.

FetchXML is a robust query language that was developed by Microsoft to enable complex operations to be performed against Dataverse data. It provides considerably more filtering options than the OData connectors, and it also allows you to see the labels for reference data such as Option Set labels or lookup columns.

You can run FetchXML queries by using the following methods:

- Dataverse Web API
- Organization Service

You can also apply a custom FetchXML filter to a lookup column within a model-driven app. This approach allows for more complex lookup scenarios that aren't easily accomplished out of the box. For more information on how to complete this process, see the [addCustomFilter Client API Reference](#).

Example FetchXML query

XML

```
<fetch mapping='logical'>
  <entity name='account'>
    <attribute name='accountid' />
    <attribute name='name' />
  </entity>
</fetch>
```

XML

```
<fetch mapping='logical'>
  <entity name='account'>
    <attribute name='accountid' />
    <attribute name='name' />
    <filter type='and'>
      <condition
        attribute='address1_stateorprovince'
        operator='eq' value='WA' />
    </filter>
  </entity>
</fetch>
```

© Copyright Microsoft Corporation. All rights reserved.

The following sample query retrieves the accountid and name columns from the account table:

If you need to filter this data, for example by state, you could do something like the following sample query:

Learning Path 3 practice labs



Introduction to developing with the Power Platform

Lab 5: Power Platform APIs

© Copyright Microsoft Corporation. All rights reserved.

As we continue to build our solution, we will now perform basic operations with the Dataverse APIs.

Summary



- Power Platform provides tools and resources to extend Power Platform solutions
- Power Platform provides a robust extensibility framework in which developers can write code to expand platform capabilities and implement functionality otherwise not available to no-code and low-code makers
- Microsoft Dataverse provides a robust API to enable interaction with its underlying data and actions

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Introduction to Microsoft Power Platform developer resources
2. Use developer tools to extend Microsoft Power Platform
3. Introduction to extending Microsoft Power Platform
4. Work with Dataverse APIs



© Copyright Microsoft Corporation. All rights reserved.